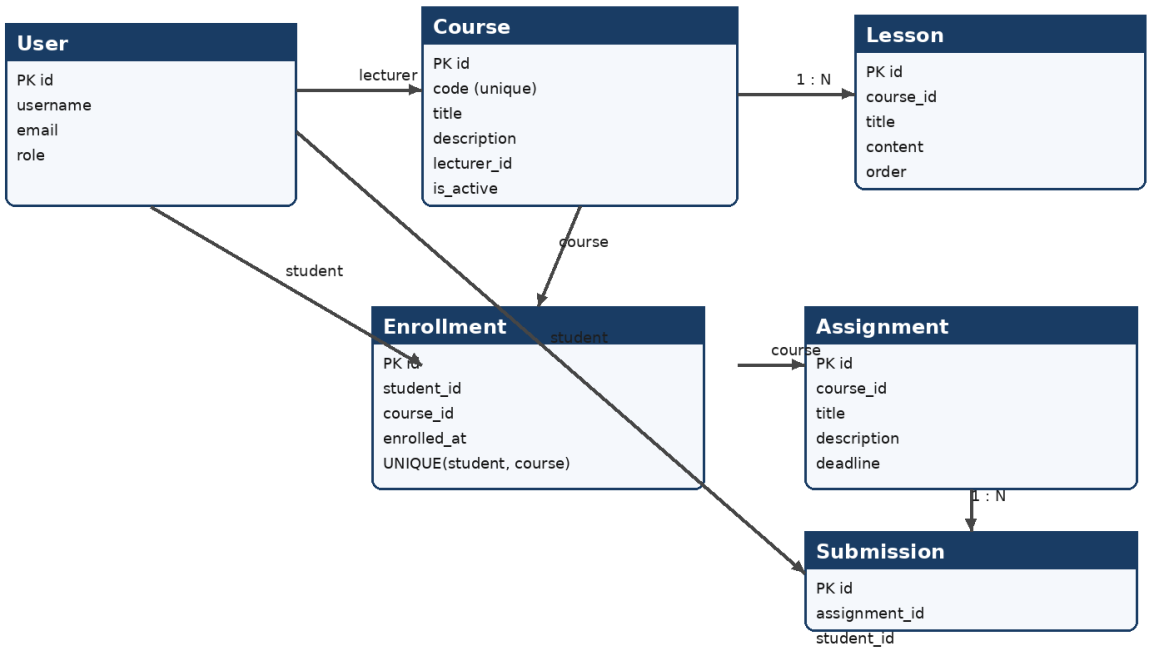
MODUL PRAKTIKUM 4

DJANGO FRAMEWORK & ORM

Docker Development Environment + Core LMS Models

Pemrograman Sisi Server — A11.64707
Suprayogi, M.Kom.

ERD Core Simple LMS — Minggu 4



Prinsip Minggu 4
Praktik terbimbing dilakukan di kelas; penugasan dikerjakan setelah konsep dan contoh utama dipahami. Target akhir bukan sekadar Django berhasil jalan, tetapi Core LMS Models dapat dimigrasikan, diuji, diperiksa melalui Admin, dan disimpan secara reproducible di GitHub.

1. Capaian Praktikum

- Menjalankan Django dari app container tanpa menginstal Django pada host.
- Mendefinisikan Custom User, Course, Lesson, Enrollment, Assignment, dan Submission.
- Membuat dan menerapkan migration ke PostgreSQL.
- Melakukan CRUD dan filtering menggunakan Django ORM melalui Django Shell.
- Menerapkan ForeignKey dan intermediate model untuk relasi domain.
- Menggunakan QuerySet traversal serta custom QuerySet/Manager sederhana.
- Mengonfigurasi Django Admin untuk data development.
- Membuat sample data idempotent dan minimal tiga unit test.
- Menguji relationship query dan mengenali N+1 sebagai jembatan menuju Minggu 5.
- Menyimpan hasil akhir dalam Git repository dan README yang dapat direproduksi.

2. Environment Praktikum

Gunakan folder kerja Minggu 4 di host Windows, dibuka dengan VS Code. Runtime Python, Django, dan PostgreSQL berada di Docker. Pada praktik kita folder kerja berada di bawah D:\minggu-4; nama folder boleh berbeda selama struktur dan perintah konsisten.

Komponen	Lokasi / cara akses
VS Code	Host Windows
Source code Django	Folder project host yang di-mount ke container app
Python + Django	Container app
PostgreSQL	Container db
Database storage	Named Docker volume
Git	Host Windows
Perintah Django	docker compose exec app python manage.py ...

Prinsip lingkungan

Mahasiswa tidak perlu menginstal Django dan PostgreSQL langsung di host untuk praktikum ini. Semua runtime aplikasi dan database dijalankan melalui Docker Compose.

Target struktur akhir

```
minggu-4/
├── manage.py
├── config/
├── users/
├── catalog/
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── .env
├── .gitignore
└── .dockerignore
```

3. Konsep yang Harus Dipahami Sebelum Coding

Apa itu Django?

Django adalah framework Python yang mempermudah pembuatan situs web dengan menggunakan bahasa pemrograman Python.

Django menangani berbagai aspek teknis yang rumit, sehingga kita dapat lebih fokus pada pengembangan aplikasi web.

Django menekankan pada penggunaan kembali komponen—yang dikenal dengan prinsip DRY (*Don't Repeat Yourself*)—serta menyediakan fitur-fitur siap pakai seperti sistem login, koneksi basis data, dan operasi CRUD (*Create, Read, Update, Delete*).

Bagaimana cara kerja Django?

Django mengikuti pola desain MVT (Model View Template).

Model - Data yang ingin disajikan, biasanya berupa data dari database.

View - Penangan permintaan (request handler) yang mengembalikan template dan konten yang relevan, berdasarkan permintaan dari user.

Template - File teks (seperti file HTML) yang berisi tata letak halaman web, beserta logika tentang cara menampilkan data tersebut.

Model

Model menyediakan data dari basis data (database).

Di Django, data disajikan melalui *Object Relational Mapping* (ORM), yaitu sebuah teknik yang dirancang untuk mempermudah pekerjaan dengan basis data.

Cara yang paling umum untuk mengambil data dari basis data adalah dengan menggunakan SQL. Salah satu kendala dalam penggunaan SQL adalah kita harus memiliki pemahaman yang cukup baik mengenai struktur basis data agar dapat mengoperasikannya.

Django, dengan ORM-nya, mempermudah komunikasi dengan basis data tanpa perlu menulis pernyataan SQL yang rumit.

Model biasanya ditempatkan di dalam berkas bernama `models.py`.

View

View adalah fungsi atau metode yang menerima permintaan HTTP sebagai argumen, mengimpor model yang relevan, menentukan data apa yang perlu dikirim ke template, lalu mengembalikan hasil akhirnya.

View biasanya ditempatkan di dalam file bernama views.py.

Template

Template adalah sebuah file yang mendefinisikan bagaimana hasil akhir harus ditampilkan.

Template sering kali berupa file .html yang berisi kode HTML untuk mengatur tata letak halaman web. Meskipun bisa juga menggunakan format file lain untuk menyajikan hasil dalam bentuk berbeda.

Django menggunakan HTML standar untuk menentukan tata letak, namun memanfaatkan tag Django untuk menambahkan logika.

URL

Django juga menyediakan cara untuk bernavigasi di antara berbagai halaman dalam sebuah situs web.

Saat user meminta suatu URL, Django meneruskan ke *view* dimana permintaan tersebut akan diarahkan.

Hal ini diatur dalam sebuah berkas bernama `urls.py`.

Project, App, Model, ORM, dan Database

Konsep	Makna dalam praktikum
Django Project	Konfigurasi dan kumpulan aplikasi untuk satu sistem.
Django App	Modul domain/fungsi, misalnya users dan catalog.
Model	Representasi data/domain yang dipetakan Django ORM ke tabel database.
ORM	Lapisan untuk membaca/menulis database melalui objek Python.
Migration	Catatan perubahan struktur database berdasarkan perubahan model.
Admin	Antarmuka development untuk memeriksa dan mengelola data.

MVT secara sederhana

```
Browser → URL → View → Model/ORM → PostgreSQL
                  ↓
              Template (jika digunakan)
```

Pada Minggu 4, pembahasan utama berada pada Model dan ORM. View/API belum menjadi fokus utama.

Mengapa Custom User ditentukan sejak awal?

Custom User harus ditentukan sebelum migration authentication pertama. Dengan demikian role ADMIN, LECTURER, dan STUDENT menjadi bagian dari model User sejak awal dan project tidak perlu melakukan perubahan authentication model setelah database sudah terisi.

Checkpoint konsep
Sebelum lanjut, mahasiswa harus dapat menjawab: app apa yang menyimpan User? app apa yang menyimpan model LMS? database apa yang dipakai? dan dari mana perintah Django dijalankan?

Domain Project latihan “Simple LMS”

- User: akun dan role dari user.
- Course: kelas mata kuliah/course yang diampu dosen/lecturer.
- Lesson: materi di dalam course.
- Enrollment: student mengikuti kelas/Course ,hubungan student dengan course.
- Assignment: tugas yang diberikan pada course.
- Submission: pengumpulan tugas oleh student.

LANGKAH 1 — Membuat Fondasi Project Docker

Gunakan folder project, misalnya D:\minggu-4. Buat file berikut di root project:

```
requirements.txt
Dockerfile
compose.yaml
.env
.dockerignore
.gitignore
```

requirements.txt

```
Django>=5.2,<6.0
psycopg[binary]>=3.2,<4.0
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["tail", "-f", "/dev/null"]
# CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

compose.yaml

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    volumes:
      - .:/app
    depends_on:
      - db
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

.env

```
DB_NAME=lms_db
DB_USER=lms_user
DB_PASSWORD=lms_password
```

.gitignore

```
db.sqlite3
.env
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.coverage
htmlcov/
```

.dockerignore

```
__pycache__
*.pyc
*.pyo
```

```
*.pyd
.venv
venv
.env
.git
.gitignore
db.sqlite3
```

Penting
File .env digunakan untuk development lokal. Jangan commit credential nyata ke repository GitHub.

LANGKAH 2 — Build & Verifikasi Container

```
docker compose build
docker compose up -d
docker compose ps
```

Pastikan service app dan db berstatus berjalan. Kemudian verifikasi versi dari dalam container:

```
docker compose exec app python --version
docker compose exec app python -m django --version
docker compose exec db psql –version
```

Masuk ke PostgreSQL :

```
docker compose exec db psql -U user-1 -d dblatihan
\conninfo
```

Untuk melihat daftar database :

```
\l
```

Untuk melihat daftar user/role PostgreSQL:

```
\du
membuat tabel:
CREATE TABLE mhs (
  nim VARCHAR(15),
  nama VARCHAR(20),
  umur INT
);
melihat daftar tabel
\dt
melihat desc tabel
\d mhs
```

keluar:

```
\q
```

LANGKAH 3 — Bootstrap Django Project dan App

Membuat Project Django

django-admin startproject nama_project

Membuat App Django

App adalah aplikasi web yang memiliki fungsi khusus dalam suatu proyek , seperti home page, contact form, atau a members database.

```
python manage.py startapp users nama_app
```

Jalankan semua perintah Django dari container app:

```
docker compose exec app django-admin startproject config .
docker compose exec app python manage.py startapp users
docker compose exec app python manage.py startapp catalog
```

Struktur awal

```
minggu-4/
├── manage.py
├── config/
├── users/
├── catalog/
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── .env
├── .gitignore
└── .dockerignore
```

Tambahkan users dan catalog ke INSTALLED_APPS pada config/settings.py. Pastikan database menggunakan PostgreSQL dari environment Docker.

```
import os
INSTALLED_APPS = [
 "django.contrib.admin",
 "django.contrib.auth",
 "django.contrib.contenttypes",
 "django.contrib.sessions",
 "django.contrib.messages",
 "django.contrib.staticfiles",
 "users",
 "catalog",
]
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ.get("DB_NAME"),
        "USER": os.environ.get("DB_USER"),
        "PASSWORD": os.environ.get("DB_PASSWORD"),
        "HOST": "db",
        "PORT": "5432",
    }
}
```

Checkpoint 1
docker compose ps menunjukkan app dan db berjalan; python --version, django --version, dan psql --version dapat dijalankan dari container.

Kemudian ubah Dockerfile menjadi sbb:

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
#CMD ["tail", "-f", "/dev/null"]
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Lakukan build ulang :

```
docker compose build app
docker compose up -d
docker compose exec app python -m django --version
```

LANGKAH 4 — Custom User Sebelum Migration Authentication

Custom User harus ditentukan sebelum migration authentication pertama. Ini menghindari kebutuhan menghapus database atau mengulang migration ketika project sudah berisi data.

users/models.py

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    class Role(models.TextChoices):
        ADMIN = "ADMIN", "Administrator"
        LECTURER = "LECTURER", "Dosen"
        STUDENT = "STUDENT", "Mahasiswa"

    role = models.CharField(
        max_length=20,
        choices=Role.choices,
        default=Role.STUDENT,
    )

    def __str__(self):
        return f"{self.username} ({self.role})"
```

config/settings.py

```
AUTH_USER_MODEL = "users.User"
```

Migration

```
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate
docker compose exec app python manage.py showmigrations
docker compose exec app python manage.py check
```

Verifikasi melalui Django Shell

```
docker compose exec app python manage.py shell

from users.models import User
User.objects.count()
exit()
```

Membuat Lecturer dan Student melalui ORM

Insert Data

```
from users.models import User
User.objects.all()
dosen = User.objects.create_user(
    username="suprayogi",
    email="suprayogi@gmail.com",
    password="12345",
    role=User.Role.LECTURER,
)
student = User.objects.create_user(
 username="fauzan",
 email="fauzan@gmail.com",
 password="12345",
 role=User.Role.STUDENT,
)
```

Add Multiple Records

```
dosen1 = User.objects.create_user(username="agus",email="agus@gmail.com",password="123",role=User.Role.LECTURER,)
dosen2 = User.objects.create_user(username="ayu",email="ayu@gmail.com",password="123",role=User.Role.LECTURER,)
dosen3 = User.objects.create_user(username="bagus",email="bagus@gmail.com",password="123",role=User.Role.LECTURER,)
dosen4 = User.objects.create_user(username="heru",email="heru@gmail.com",password="123",role=User.Role.LECTURER,)
list_dosen = [dosen1, dosen2, dosen3, dosen4]
```

```
for x in list_dosen:
   x.save()

User.objects.all().values()
```

Update Data

```
from catalog.models import User
x = User.objects.get(username=”Fauzan”)
x.username = "Fauzan Arif"
x.save()
User.objects.all().values()
print(user.dict)
```

Delete Data

```
x = User.objects.get(username=”Budiharjo”)
x.delete()
```

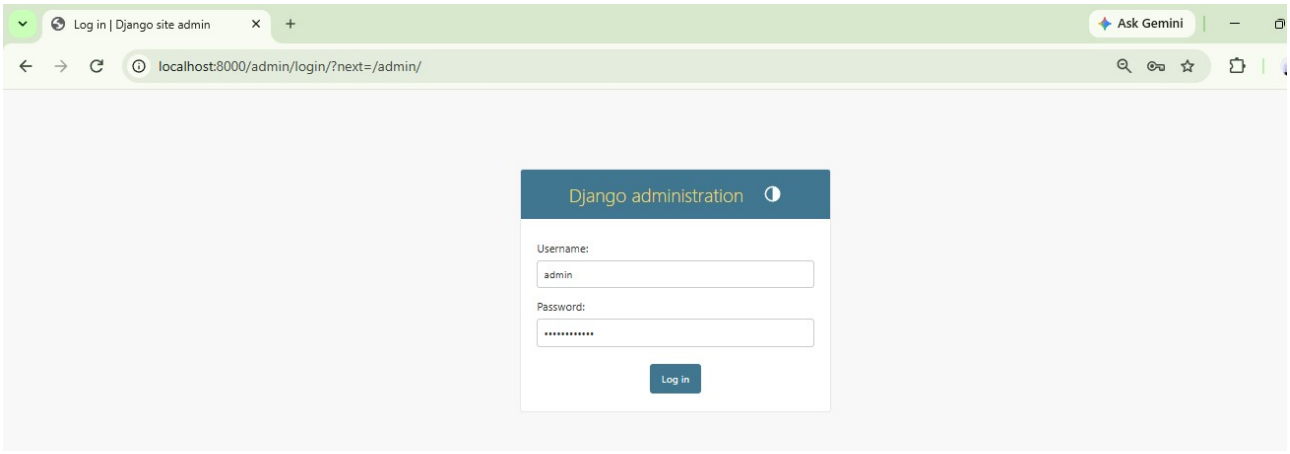
LANGKAH 5–6 — Superuser, Lecturer, dan Student

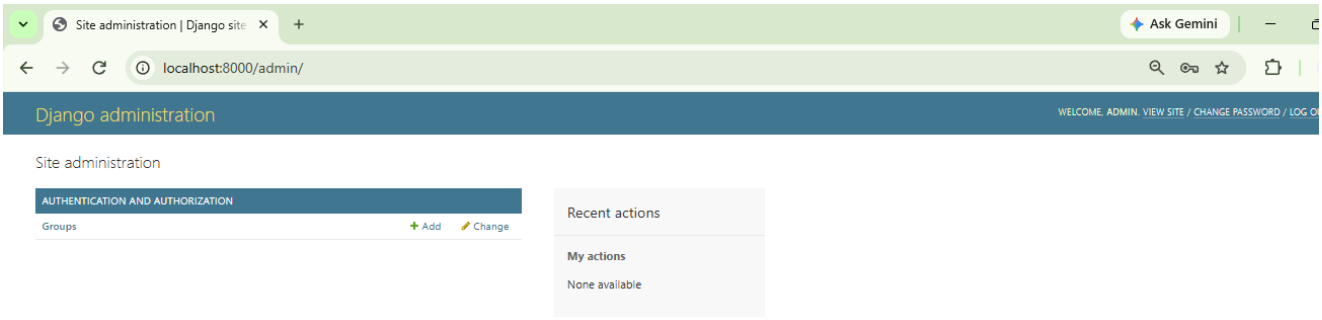
Membuat Superuser

```
docker compose exec app python manage.py createsuperuser
```

Untuk praktik, username yang digunakan adalah admin. Login Django Admin menggunakan username superuser dan password yang dibuat.

http://localhost:8000/admin/





Pastikan superuser ber-role ADMIN

```
docker compose exec app python manage.py shell
```

```
from users.models import User
admin = User.objects.get(username="admin")
admin.role = User.Role.ADMIN
admin.is_staff = True
admin.is_superuser = True
admin.save()

User.objects.values("username", "email", "role", "is_staff", "is_superuser")
```

Catatan praktik
Role default User adalah STUDENT. Karena itu role superuser perlu diverifikasi dan, jika perlu, diubah menjadi ADMIN agar data development konsisten.

LANGKAH 7 — Course melalui Django ORM

Course memiliki lecturer sebagai foreign key. Dalam domain ini lecturer adalah User dengan role LECTURER.

Update file catalog\models.py

```
from django.conf import settings
from django.db import models


class Course(models.Model):
    code = models.CharField(max_length=20, unique=True)
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    lecturer = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="courses_taught",
        limit_choices_to={"role": "LECTURER"},
    )
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["code"]

    def __str__(self):
        return f"{self.code} - {self.title}"
```

Migration dan data awal

```
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate
```

Cek apakah course sudah masuk PostgreSQL

```
docker compose exec app python manage.py shell
```

```
from catalog.models import Course
from users.models import User

lecturer = User.objects.get(username="suprayogi")
course = Course.objects.create(
    code="PSS01",
    title="Pemrograman Sisi Server",
    description="Django Framework dan ORM",
    lecturer=lecturer,
)
print(course.code, course.lecturer.username)
```

Konsep yang perlu dipahami

- unique=True pada code mencegah dua course memiliki kode yang sama.
- on_delete=PROTECT menjaga Course agar tidak terhapus hanya karena lecturer dihapus.
- related_name="courses_taught" memungkinkan traversal dari User ke Course.
- limit_choices_to membatasi pilihan lecturer pada user ber-role LECTURER di konteks form/admin.

Checkpoint 3
Course PSS01 berhasil dibuat dan course.lecturer.username menampilkan nama lecturer.

LANGKAH 8 — Lesson dan ForeignKey

catalog/models.py :

```
class Lesson(models.Model):
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="lessons",
    )
    title = models.CharField(max_length=200)
    content = models.TextField()
    order = models.PositiveIntegerField(default=1)

    class Meta:
        ordering = ["course", "order"]
        constraints = [
            models.UniqueConstraint(
                fields=["course", "order"],
                name="unique_lesson_order_per_course",
            )
        ]

    def __str__(self):
        return f"{self.course.code} - {self.title}"
```

Migration dan data

```
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate
```

```
docker compose exec app python manage.py shell
```

```
course = Course.objects.get(code="PSS01")
Lesson.objects.create(course=course, title="Pengenalan Backend", order=1)
Lesson.objects.create(course=course, title="Pola Arsitektur Backend", order=2)
Lesson.objects.create(course=course, title="Kontaienerisasi dengan Docker", order=3)
course.lessons.all()
```

Traversal relationship

```
lesson = Lesson.objects.get(order=1)
```

```
lesson.course
course.lessons.all()
```

```
for lesson in course.lessons.all():
```

```
    print(lesson.order, lesson.title)
```

Relasi ini berarti satu Course dapat memiliki banyak Lesson. Field course pada Lesson menyimpan foreign key ke Course, sedangkan related_name="lessons" menyediakan akses balik dari Course.

Checkpoint 4
Pada praktik, Course PSS01 menghasilkan tiga Lesson. Coba akses lesson.course dan course.lessons.all() untuk memahami traversal dua arah.

LANGKAH 9 — Enrollment sebagai Intermediate Model

Enrollment merepresentasikan hubungan Student dan Course. Secara konseptual, seorang student dapat mengikuti banyak course dan sebuah course dapat diikuti banyak student. Karena hubungan ini memiliki data tambahan enrolled_at, kita gunakan model intermediate.

catalog/models.py :

```
class Enrollment(models.Model):
    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="enrollments",
        limit_choices_to={"role": "STUDENT"},
    )
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="enrollments",
    )
    enrolled_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["student", "course"],
                name="unique_student_course",
            )
        ]

    def __str__(self):
        return f"{self.student.username} -> {self.course.code}"
```

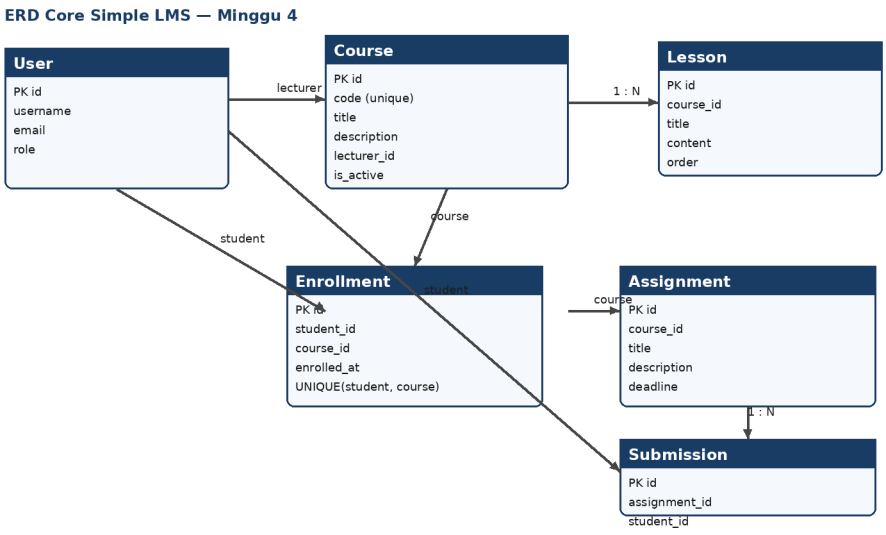
Migration dan menguji duplikasi

```
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate
```

```
docker compose exec app python manage.py shell
```

```
student = User.objects.get(username="fauzan")
course = Course.objects.get(code="PSS01")
Enrollment.objects.create(student=student, course=course)
Enrollment.objects.all()
```

Coba membuat enrollment fauzan + PSS01 untuk kedua kalinya. Constraint unique_student_course harus mencegah duplikasi student-course.



LANGKAH 10 — Django Admin

Django Admin digunakan sebagai alat pemeriksaan data development. Daftarkan model inti agar hasil migration dan ORM dapat diverifikasi secara visual.

catalog/admin.py

```
from django.contrib import admin
from .models import (
    Course,
    Lesson,
    Enrollment,
)
@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ("code", "title", "lecturer", "is_active")
    search_fields = ("code", "title")
    list_filter = ("is_active",)

@admin.register(Lesson)
class LessonAdmin(admin.ModelAdmin):
    list_display = ("course", "title", "order")
    search_fields = ("title", "course__title")
    list_filter = ("course",)

@admin.register(Enrollment)
class EnrollmentAdmin(admin.ModelAdmin):
    list_display = ("student", "course", "enrolled_at")
    search_fields = ("student__username", "course__title")
    list_filter = ("course",)
```

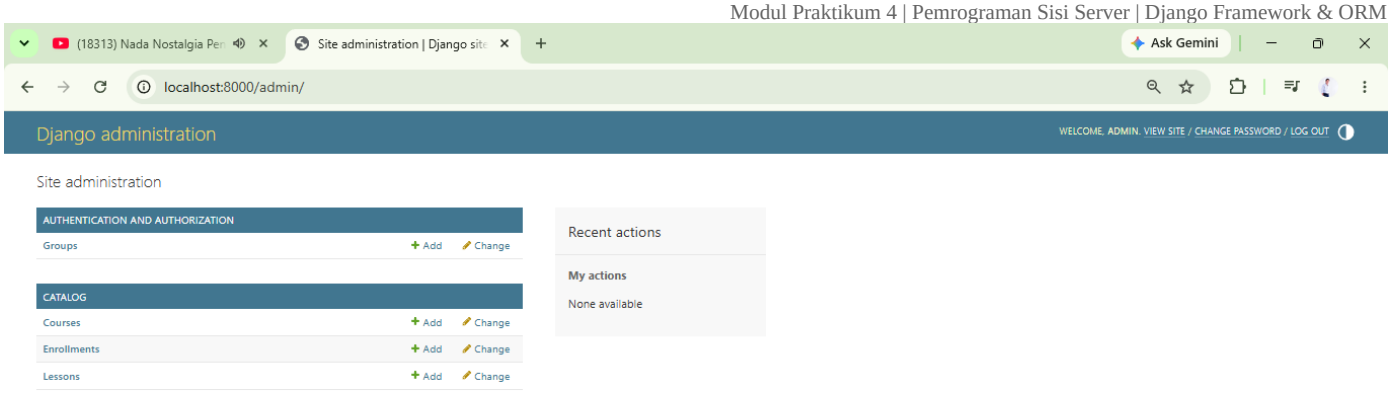
Yang diperiksa melalui Admin

- Users dan role pengguna.
- Courses dan lecturer.
- Lessons pada setiap course.
- Enrollment dan relasi student-course.

Checklist Admin

1. Jalankan docker compose up -d jika container belum aktif.
2. Buka http://localhost:8000/admin/.
3. Login menggunakan username superuser.
4. Periksa apakah model yang sudah didaftarkan muncul di sidebar.
5. Buka data Course PSS01 dan pastikan lecturer benar.

Catatan
Admin adalah alat verifikasi development. Fokus Minggu 4 tetap pada model dan ORM, bukan membangun tampilan administrasi production.


(18313) Nada Nostalgia Per
Site administration | Django site
Ask Gemini
localhost:8000/admin/
Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups + Add Change
CATALOG
Courses + Add Change
Enrollments + Add Change
Lessons + Add Change
Recent actions
My actions
None available

LANGKAH 11 — Assignment dan Submission

Assignment berelasi langsung dengan Course. Submission berelasi dengan Assignment dan Student. Dengan demikian, Assignment tidak perlu berelasi langsung dengan Lesson pada model inti yang digunakan di praktikum ini.

catalog/models.py

```
class Assignment(models.Model):
    course = models.ForeignKey(
        Course, on_delete=models.CASCADE, related_name="assignments"
    )
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    deadline = models.DateTimeField()

class Submission(models.Model):
    assignment = models.ForeignKey(
        Assignment, on_delete=models.CASCADE, related_name="submissions"
    )
    student = models.ForeignKey(
        "users.User", on_delete=models.CASCADE, related_name="submissions"
    )
    content = models.TextField(blank=True)
    submitted_at = models.DateTimeField(auto_now_add=True)
    grade = models.DecimalField(max_digits=5, decimal_places=2, null=True, blank=True)
```

tambahkan pada catalog/admin.py

```
from django.contrib import admin
from .models import (
    Course,
    Lesson,
    Enrollment,
    Assignment,
    Submission,

)
admin.site.register(Assignment)
admin.site.register(Submission)

@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ("code", "title", "lecturer", "is_active")
    search_fields = ("code", "title")
    list_filter = ("is_active",)

@admin.register(Lesson)
class LessonAdmin(admin.ModelAdmin):
    list_display = ("course", "title", "order")
    search_fields = ("title", "course__title")
    list_filter = ("course",)

@admin.register(Enrollment)
class EnrollmentAdmin(admin.ModelAdmin):
    list_display = ("student", "course", "enrolled_at")
    search_fields = ("student__username", "course__title")
    list_filter = ("course",)
```

Migration dan contoh data

```
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate
docker compose exec app python manage.py shell


from django.utils import timezone
from datetime import timedelta
from catalog.models import Course
from users.models import User
from catalog.models import Assignment, Submission

course = Course.objects.get(code="PSS01")
student = User.objects.get(username="fauzan")

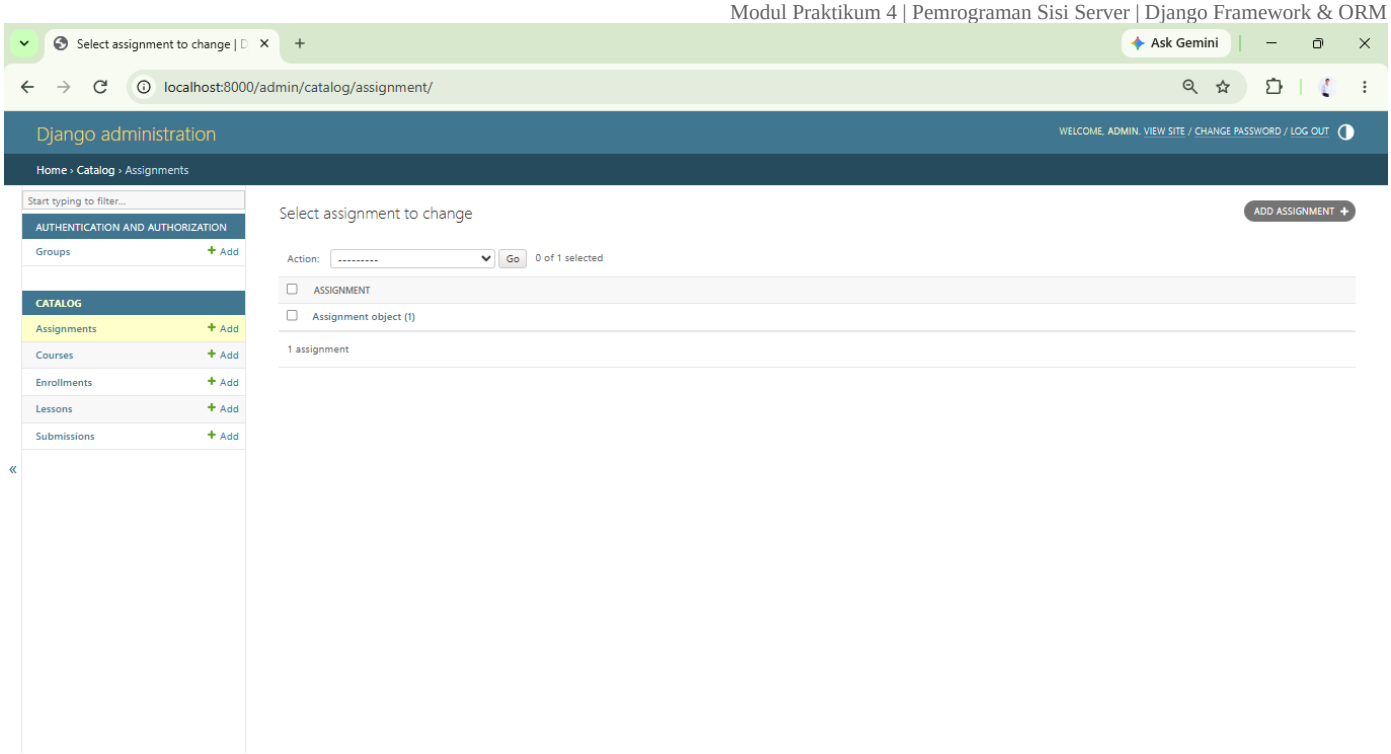
assignment = Assignment.objects.create(
    course=course,
    title="Tugas Django ORM",
    description="Implementasikan model dan relationship menggunakan Django ORM.",
    deadline=timezone.now() + timedelta(days=7),
)

submission = Submission.objects.create(
    assignment=assignment,
    student=student,
    content="Saya telah menyelesaikan implementasi Django ORM dan relationship.",
    grade=87.50,
)
```

Checkpoint 5
Periksa /admin/catalog/assignment/ dan /admin/catalog/submission/. Assignment dan Submission harus dapat ditampilkan melalui Admin.

```
Submission.objects.all()
Assignment.objects.all()
assignment = Assignment.objects.get(title="Tugas Django ORM")
assignment
assignment.title
assignment.description
assignment.deadline

submission = Submission.objects.first()
submission
submission.content
submission.grade
assignment.submissions.all()
student = User.objects.get(username="fauzan")
User.objects.all()
student
Submission.objects.all()
```


Select assignment to change | D
Ask Gemini
localhost:8000/admin/catalog/assignment/
Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Catalog › Assignments
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
CATALOG
Assignments + Add
Courses + Add
Enrollments + Add
Lessons + Add
Submissions + Add
Select assignment to change
ADD ASSIGNMENT +
Action: --------- Go 0 of 1 selected
ASSIGNMENT
Assignment object (1)
1 assignment

LANGKAH 12 — QuerySet dan Relationship Traversal

Gunakan Django Shell untuk membuktikan bahwa relationship dapat ditelusuri tanpa menulis SQL manual.

```
docker compose exec app python manage.py shell
```

```
from catalog.models import Course
course = Course.objects.get(code="PSS01")
print(course.code)
for lesson in course.lessons.all():
    print("-", lesson.title)
```

select_related sebagai pengantar optimasi

```
from django.db import connection, reset_queries
from catalog.models import Course

reset_queries()
courses = Course.objects.all()
for course in courses:
 print(course.code, course.lecturer.username)

print(len(connection.queries))


reset_queries()
courses = Course.objects.select_related("lecturer")
for course in courses:
 print(course.code, course.lecturer.username)

print(len(connection.queries))
```

Pada praktik kita, satu Course menghasilkan 2 query tanpa select_related dan 1 query dengan select_related. Ini merupakan pengantar langsung ke materi optimasi query Minggu 5.

Query 1 → ambil Course

Query 2 → ambil User/lecturer

```
relasi Course dengan lecturer adalah OneToOne:
```

select_related:

Course
└── lecturer
↓
User

prefetch_related untuk relasi reverse:

Untuk relasi dari **Course → banyak Lesson** sebaiknya menggunakan **prefetch_related.**

Contoh query biasa tanpa **prefetch_related.**

```
from django.db import connection, reset_queries
from catalog.models import Course

reset_queries()

courses = Course.objects.all()

for course in courses:
    print(course.code)

    for lesson in course.lessons.all():
        print("-", lesson.title)

print(len(connection.queries))
```

Contoh query biasa menggunakan **prefetch_related :**

```
reset_queries()

courses = Course.objects.prefetch_related("lessons")
```

```
for course in courses:
    print(course.code)

    for lesson in course.lessons.all():
        print("-", lesson.title)

print(len(connection.queries))
```

```
relasi Course dengan Lesson adalah OneToMany:
Course
   │
   ├── Lesson 1
   ├── Lesson 2
   └── Lesson 3
```

Relasi	Teknik	Fungsi
Course → Lecturer	`select_related()`	ForeignKey / OneToOne
Course → Lessons	`prefetch_related()`	Reverse ForeignKey / relasi banyak

Batas materi
Eksperimen query pada Minggu 4 bertujuan mengenalkan pola. Analisis N+1 secara sistematis, EXPLAIN, indexing, benchmark, dan optimasi dataset besar menjadi fokus Minggu 5.

LANGKAH 13 — Git & GitHub Setelah Implementasi Stabil

Pada praktik Minggu 4, Git diterapkan setelah implementasi lokal, test, dan verifikasi selesai. Urutan ini dipertahankan agar commit merepresentasikan satu milestone yang dapat dijelaskan.

```
git status --short
git init
git status
```

Pastikan .gitignore sudah ada dan .env tidak masuk repository. Kemudian lakukan commit milestone:

```
git add .
git status
git commit -m "feat: complete week 4 django orm core lms models"
```

Menggunakan branch main

```
git branch
git branch -M main
git log --oneline -1
```

Hubungkan GitHub dan push

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git remote -v
git push -u origin main
```

Makna prefix commit

Prefix	Makna umum
feat:	Menambahkan fitur/perubahan fungsional baru
fix:	Memperbaiki bug
docs:	Perubahan dokumentasi
test:	Menambahkan/memperbaiki pengujian
refactor:	Perubahan struktur kode tanpa mengubah tujuan fungsional

Contoh praktik

Commit utama Minggu 4 menggunakan "feat: complete week 4 django orm core lms models". Setelah README ditambahkan, commit berikutnya menggunakan "docs: add week 4 README".